FINAL YEAR DESIGN PROJECT REPORT

Intelligent Shopping Cart for Enhanced Theft Prevention and Inventory Tracking

Supervised by

Lt Cdr Dr. Gul Shahzad

PAKISTAN NAVY ENGINEERING COLLEGE
NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY
PAKISTAN

(Spring 2025)

Team Members	**Signature**
Hadia Zulfiqar	_____
Taha Hussain Rao	_____
Mohammed Umar	_____

Supervisor **Signature**

Lt Cdr Dr. Gul Shahzad _____

Checked By **Signature**

_____

_____

_____

_____

Date:

_____

Plagiarism report attached.

ACKNOWLEDEGMETS

`We would like to express our utmost gratitude and appreciation to all those who have supported and assisted us throughout the course of this project and the writing of this research report. We extend our sincere thanks to our supervisor, Lt Cdr Dr. Gul Shahzad, for his insightful guidance, valuable feedback, and unwavering support. His encouragement and technical expertise were instrumental to the successful completion of this project. We are also grateful to our families and friends for their patience, motivation, and constant encouragement during challenging phases of this work. Our heartfelt thanks go out to our colleagues and classmates for their valuable input, constructive criticism, and collaboration throughout the development and testing process. Lastly, we acknowledge the authors and researchers whose work we have referenced, as their contributions, laid the foundation for our research and development. This project would not have been possible without the collective support we received from all these individuals.

TABLE OF CONTENTS

1. CHAPTER: INTRODUCTION 9

1.1 Smart Retail Solutions 9

1.1.1 Introduction to Smart Retail Solutions 9

1.1.2 Importance of Smart Retail Solutions 9

1.1.3 Approaches to Deploy Smart Retail Frameworks 9

1.2 Consumer Insights And Market Analysis 10

2. CHAPTER: LITERATURE REVIEW 12

2.1 Existing Solutions 12

2.1.1 Smart Retail Stores 12

2.1.2 Smart Carts 12

2.2 Research 13

2.2.1 RFID Solutions 13

2.2.2 Object Detection Models 14

2.2.3 Mobile Applications 15

2.2.4 Weight Sensors 15

3. CHAPTER: PROBLEM DEFINITION 18

4. CHAPTER: METHODOLOGY 19

4.1 Hardware Integration 19

4.2 Software Development 20

4.3 Anti-Theft System 21

4.4 Billing 21

4.5 Testing And Validation 22

5. CHAPTER: DETAILED CIRCUIT & SYSTEM DESIGN 24

5.1: System Design 24

5.1.1: Billing System 24

5.1.2: Weight Sensor 25

5.1.4: Battery Management System 26

5.1.5: Barcode Scanning 26

5.1.3: Inventory Management 27

5.1.6: User Interface 28
5.2: System Integration 28
6. CHAPTER: COMPONENTS SPECIFICATIONS 30
6.1 Billing System 30
6.2 Weight Sensor 30
6.4 Battery Management System 31
6.5 Barcode Scanning 31
6.6 User Interface 32
7. CHAPTER: IMPLEMENTATION AND TESTING 34
7.1 Weight Sensor Parameter Tuning And Stability Testing 34
7.2 Power Consumption Analysis 37
7.3 Camera Selection And Justification 38
7.4 Hardware Integration and Enclosure Design 40
8. CHAPTER: RESULTS AND DISCUSSION 43
8.1 Graphical User Interface (GUI) 43
8.2 Weight Sensor Functionality 47
8.3 Camera and Barcode Scanning Performance 48
8.5 Observed System Performance 49
9. CHAPTER: FINANCIAL DETAILS 51
9.1 Budget Allocation 51
9.2 Cost Optimization Strategies 52
9.3 Justification Of High-Cost Items 52
9.3 Future Cost Reduction 53
10. Chapter: CONCLUSION AND FUTURE WORK 54
10.1 Conclusion 54
10.2 Future Directions 55

TABLE OF FIGURES

Figure 1-1.1 Issues Faced During Checkout 11
Figure 1.1.2 Time Spent on Checkouts 11
Figure 4.1 Flow Diagram of Proposed Methodology 23
Figure 5.1: Block diagram of System. All the systems are connected to Raspberry Pi. 24
Figure 5.2 Circuit design of Billing System 25
Figure 5.3 Complete System Design 29
Figure 6.1 MFRC522 RFID Reader 30
Figure 6.2 Avery LM10 31
Figure 6.3 Raspberry Pi Camera Module 3. 32
Figure 6.4 7-inch HDMI Capacitive Touch Screen LCD Display 32
Figure 7.1 Distortion 39
Figure 7.2 Clarity 40
Figure 7.3 Edge Detection 40
Figure 7.4 Front view of the enclosure with touchscreen and camera module. 41
Figure 7.5 Top view of the enclosure mounted on the shopping cart handle 41
Figure 7.6 Internal arrangement of hardware components in the smart cart enclosure. 42
Figure 7.7 Complete setup of the Smart Cart 42
Figure 8.1 Smart Cart UI Workflow Overview 43
Figure 8.2 Payment Method Selection Screen 44
Figure 8.3 PIN Entry Interface for Secure Payment 44
Figure 8.4 Product Scanning and Cart Interface 45
Figure 8.5 Overweight Detection and Alert Message 45
Figure 8.6 Frame showing total of the bill 46
Figure 8.7 Thank-you screen (2-second display before GUI reset) 46
Figure 8.8 Minimum weight detected by the sensor 47
Figure 8.9 Load cell setup in cart and LM10 display in action. 48
Figure 8.10 Barcode scanning test using Raspberry Pi Camera Module 3 under varying lighting conditions. 48
Figure 8.11 Close-up of barcode decoding process with Pyzbar library 49

TABLE OF TABLES

Table 2.1 Comparison of RFID Technology and Traditional Barcodes (Yewatkar, 2016) 14
Table 5.1 Camera Modules Comparison Table 26
Table 7.1 LM10 Weighing Indicator parameters 35
Table 7.2 Comparison between power bank and lead acid battery 37
Table 9.1: Table of Budget Allocation for Smart Cart 51

ABSTRACT

The Smart Shopping Cart project is designed to modernize and simplify the shopping experience by automating item detection, identification, and inventory management using Edge AI. Traditional retail processes rely heavily on manual checkout, which can lead to errors, long queues, and losses due to theft. This project aims to create a cost-effective smart cart that uses barcode scanners and weight sensors to validate items in real-time as they are added to or removed from the cart, thus preventing theft and improving efficiency.

The cart also communicates with the store's inventory system to keep stock levels updated in real time. Unlike RFID systems that are expensive and difficult to implement on a large scale, this solution adapts to readily available technology (barcode and weight sensors), making it accessible for many retailers. The proposed system is scalable, and its modular design will allow future upgrades, such as the integration of object recognition and advanced AI models. In conclusion, this project addresses major pain points for retailers, including theft prevention, inventory management, and a smoother checkout process.

1. CHAPTER: INTRODUCTION

1.1 SMART RETAIL SOLUTIONS

1.1.1 Introduction to Smart Retail Solutions

Smart retail solutions integrate modern technologies like IoT, AI, and cloud computing to enhance the shopping experience and streamline retail operations. These systems aim to improve the efficiency of both customers and retailers by automating processes such as billing, inventory management, and customer interactions. Solutions like smart shopping carts, RFID-based tracking, and neural network-powered product recognition enable retailers to offer personalized recommendations, reduce checkout times, and ensure better resource management. The growing interest in e-commerce has prompted traditional retailers to adopt smart retail technologies to remain competitive and improve customer satisfaction (Hussain & Rumy, 2021; Kumar, 2017).

1.1.2 Importance of Smart Retail Solutions

Smart retail frameworks play a crucial role in reshaping the retail environment by enhancing customer experience and operational efficiency. By leveraging technologies like RFID, ZigBee, and IoT, these frameworks reduce waiting times at checkout, provide real-time inventory tracking, and allow personalized marketing based on user data. These systems also help retailers gain insights into consumer behavior and optimize store layouts or promotions accordingly. The ability to reduce human errors, improve security, and offer seamless transactions further highlights the significance of smart retail frameworks in the modern retail landscape (Kumar, 2017; Kowshika, 2021).

1.1.3 Approaches to Deploy Smart Retail Frameworks

Implementing smart retail frameworks requires a structured approach, beginning with integrating RFID tags for product tracking and smart shopping carts for automated billing. Retailers can adopt IoT solutions to connect various in-store systems like

inventory management and customer engagement tools. Technologies such as neural networks and edge computing devices, like Intel's Neural Compute Stick, can further enhance real-time decision-making by recognizing products through cameras instead of barcodes. Additionally, mobile applications that sync with these frameworks can allow customers to shop, track purchases, and make payments seamlessly. Examples like Amazon Go, which uses AI-powered cameras and sensors for a cashier-less shopping experience, showcase the future of smart stores. Furthermore, smart mobile applications enable features such as virtual shopping lists, personalized recommendations, and contactless payment options. These solutions significantly enhance customer convenience and operational efficiency. Their technical implementations are discussed in detail in the literature review (Kowshika, 2021; Belibov & Tudose, 2019).

1.2 Consumer Insights And Market Analysis

The demand for Smart Shopping Systems has significantly increased in recent years due to advancements in technology and the growing need for efficiency in the retail sector. According to a **ResearchAndMarkets.com** report, the global Smart Shopping Cart market is expected to grow substantially from 2021 to 2026, largely driven by the cumulative impact of the COVID-19 pandemic and an increasing preference for contactless, automated shopping experiences. The rise in consumer preference for seamless, quick, and personalized shopping experiences is contributing to the growth of smart retail solutions such as AI-powered carts, self-checkout systems, and predictive analytics in consumer behavior (*Smart Shopping Cart*, 2025).

Additionally, innovations in real-time data collection and analysis are improving the efficiency of smart shopping systems, further pushing their adoption across different retail segments. Consumers are increasingly looking for solutions that save time, reduce in-store friction, and offer tailored shopping experiences, all of which are integral features of smart shopping systems. A market report by **Business Wire** highlights that the demand for these technologies will continue to grow as businesses and consumers alike seek greater convenience and innovation in retail environments (*Global Smart Shopping Cart*, 2021).

To gauge public interest and response to smart shopping systems, a survey was conducted. The survey aimed to understand consumer perceptions, willingness to use such technology, and potential concerns about the transition to smart retail. The responses (Figure 1.1 and 1.2) provided valuable insights into how consumers view the future of smart retail and their openness to adopting these innovations.

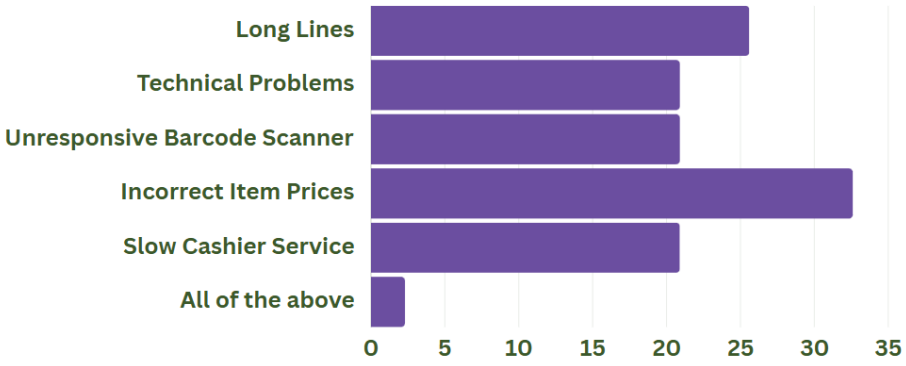


Figure 1-1.1 Issues Faced During Checkout

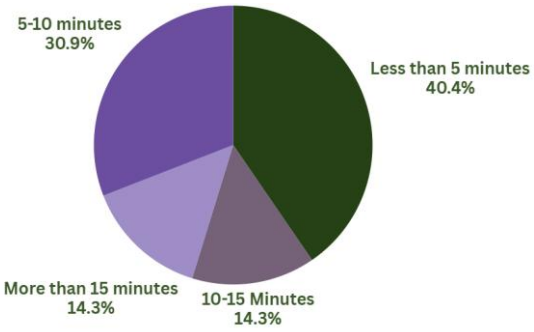


Figure 1.1.2 Time Spent on Checkouts

2. CHAPTER: LITERATURE REVIEW

2.1 EXISTING SOLUTIONS

2.1.1 Smart Retail Stores

The rise of smart retail stores, such as Amazon Go and BingoBox, has set a new standard for convenience and automation in the retail industry. **Amazon Go** stores operate using a combination of computer vision, deep learning, and sensor fusion to create a completely cashier-less shopping experience. Customers can walk into the store, pick up items, and walk out without waiting in line. The system automatically detects which products are taken from or returned to shelves and keeps track of them in a virtual cart. Once the customer leaves, the system charges them automatically using their Amazon account.

Similarly, **BingoBox**, a Chinese retail innovation, operates fully automated convenience stores without any human staff. Using RFID tags for item tracking and mobile payment integration, customers scan their items and pay via an app. These innovations represent the future of retail, where automation, AI, and IoT converge to create seamless, efficient shopping experiences with minimal human intervention.

2.1.2 Smart Carts

Several smart shopping cart models are already being deployed in retail environments, aiming to bring the smart store experience directly to individual shopping carts. One example is the **Cart**, which integrates AI, computer vision, and a touchscreen to allow customers to scan items as they shop. The cart's sensors automatically detect items placed in it, calculate the total, and allow customers to pay directly through the cart without needing to wait at a checkout line.

Another notable system is the **Instacart Caper AI Cart**, which uses cameras and sensors to recognize products in real-time, eliminating the need for scanning barcodes. These carts are designed to enhance the convenience of shopping while

minimizing friction at checkout. Several retailers have also experimented with carts that include weight sensors and real-time price displays, streamlining the shopping experience by providing customers with updated totals as they shop.

2.2 RESEARCH

2.2.1 RFID Solutions

The paper by Yewatkar *et al.* (2016) presents a smart shopping cart system that leverages both RFID and ZigBee technology to automate the billing process, provide product recommendations, and implement theft prevention measures. Each product in the system is tagged with an RFID chip, which the shopping cart can read to track items as they are placed inside. This eliminates the need for manual scanning at checkout, significantly reducing the time spent in line. Additionally, the system incorporates an anti-theft mechanism by placing an RFID reader at the store's exit, ensuring that all products leaving the store are properly accounted for. The paper emphasizes the superiority of RFID technology over traditional barcode systems, highlighting its ability to improve the overall shopping experience through faster transactions, more accurate inventory management, and a streamlined checkout process (Yewatkar, 2016).

Kumar *et al.* (2017) introduced another approach to smart shopping carts, focusing on reducing checkout times and assisting customers in managing their budgets. Their system is based on an Arduino microcontroller integrated with RFID and XBee modules, allowing users to scan products as they shop. The total cost of the selected items is displayed in real time on an LCD screen, giving customers a clear view of their spending. The system also features the ability to remove previously scanned items, which provides flexibility and ensures a more efficient shopping experience. By automating the process of tracking purchases, this system simplifies the customer journey and reduces the likelihood of errors at checkout (Kumar *et al*, 2017).

Table 2.1 Comparison of RFID Technology and Traditional Barcodes (Yewatkar, 2016)

FEATURES	RFID	BARCODE
Read Rate	High throughput. Multiple tags can be read simultaneously.	Very low throughput. Tags can only read one at a time
Line of Sight	Not required	Required
Read/Write Capability	Ability to read, write, modify, and update.	Ability to read items and nothing else.
Durability	High. Much better protected	Low. Easily damaged, cannot be read if dirty or greasy.
Security	High. Hard to replicate. Data can be encrypted	Low. Much easier to reproduce or counterfeit.
Event Triggering	Can be used to trigger certain events	Not capable.

2.2.2 Object Detection Models

Belibov and Tudose (2019) propose a smart shopping cart using Intel's Neural Compute Stick 2 to enhance product recognition through a neural network. This design aims for modularity and simplicity, allowing real-time product detection using a camera.

The solution runs locally on the cart without the need for network connectivity, addressing the limitations of existing RFID-based solutions (Belibov, 2019).

Jadhav *et al*. (2024) explored the potential of **artificial intelligence (AI)** in smart cart systems. Their work highlights the use of the **YOLOv8 model** for barcode recognition, paired with the Pyzbar library for decoding. This combination enables the system to efficiently scan products, automate billing, and improve accuracy. The authors propose a shift from barcode to QR code technology, offering enhanced security and usability. Their smart cart system aims to replace traditional checkout processes, enabling customers to have a smoother and more efficient shopping experience.

2.2.3 Mobile Applications

Kowshika *et al.* (2021) focus on integrating IoT with smart shopping trolleys. Their system uses RFID tags and a mobile application to automate billing and prevent theft using load cells. The trolley also provides users with an option to pay via mobile or QR code, further enhancing the convenience of shopping without waiting in queues.

2.2.4 Weight Sensors

Smart shopping carts represent an emerging technological innovation designed to enhance the shopping experience by integrating advanced tools such as weight sensors, Internet of Things (IoT) systems, and automated billing features. These systems are developed to improve the efficiency of the shopping process, reduce checkout times, and minimize errors in billing, all while providing a seamless, automated shopping experience for customers.

Weight sensor-based systems are a key feature of smart shopping carts, as discussed by Rajithkumar *et al*. (2018). These systems rely on load cells to verify the real-time weight of items placed in the cart. When a product is scanned, the system cross-references the scanned data with the actual weight to ensure accuracy, preventing theft and avoiding billing errors. This system is not only efficient but also highly

accurate, with load cells offering a remarkable 98.98% accuracy rate. By eliminating the need for manual verification at checkout, the weight-sensor-based smart shopping carts significantly streamline the shopping process, minimizing human error and improving overall billing accuracy.

Chandan *et al*. (2020) further advanced the concept of smart shopping carts by introducing a comprehensive system that integrates Raspberry Pi, barcode scanning, and load cells for automated billing. In this system, customers scan products as they add them to the cart, with the load cells ensuring that the actual weight of the product matches the stored values in the database. Any discrepancies between the scanned weight and the actual weight trigger an alert, ensuring accurate billing. The system is also equipped with an LCD display that provides the customer with detailed product information such as the name, price, and weight of the item, enhancing transparency during the shopping experience. Furthermore, a buzzer alerts users to any mismatch between the scanned item and its actual weight, preventing potential errors.

In addition to these features, Chandan *et al*.'s smart shopping trolley system sends the final bill directly to the customer's email and the billing counter via Wi-Fi, greatly reducing wait times at checkout. This integration of technology improves the shopping experience by eliminating long lines and offering customers real-time access to their total costs. However, despite its numerous advantages, the system does come with some challenges. The initial installation cost of such smart shopping carts is relatively high, making it a significant investment for retailers. Moreover, the system is vulnerable to certain security risks, such as customers replacing scanned items with similarly weighted products before placing them in the cart, which can lead to billing inaccuracies. This highlights the need for further development in terms of security and cost-effectiveness to make these smart shopping carts a viable option for widespread retail use.

The incorporation of **edge intelligence** and **IoT technology**, as demonstrated by Hossain and Rumy (2021), takes smart carts a step further. Their "Edge Cart" model

processes data locally using edge computing, reducing latency and enhancing real-time item identification. The system also integrates mask surveillance to promote health safety in retail spaces, demonstrating a dual-purpose application of the technology.

Despite these advancements, challenges such as cost and system misuse persist. The high expense of smart cart deployment, particularly in developing markets, limits widespread use. However, the combination of weight sensors, edge computing, and wireless communication systems shows great potential for transforming retail experiences (Hossain and Rumy, 2021).

3. CHAPTER: PROBLEM DEFINITION

Traditional retail environments face several inefficiencies that affect both store operations and customer satisfaction. One of the primary challenges is the **manual checkout process**, which leads to long wait times, human errors in billing, and increased operational costs. In addition, **theft prevention** remains a significant concern, with retailers losing millions annually due to shoplifting or accidental undercharging at checkout counters. Current anti-theft measures are often either labor-intensive or costly, as seen in RFID-based systems, which require significant infrastructure investments and are not widely adopted by smaller retailers.

Moreover, inventory management in traditional stores is also problematic, with delays in stock updates often resulting in out-of-stock situations or overstocking, both of which impact profitability. There is a need for a **cost-effective, scalable solution** that can automate item identification, improve billing accuracy, reduce checkout times, and minimize theft—all while providing real-time inventory management for the store.

This project aims to address these challenges by developing a **smart shopping cart** system. By integrating **barcode scanning, weight sensors**, and **edge computing**, the smart cart will automate the checkout process, provide real-time item verification, and detect any discrepancies between scanned items and their actual weights. This system not only prevents theft but also optimizes the overall shopping experience for customers, offering a more efficient, secure, and streamlined retail process.

4. CHAPTER: METHODOLOGY

The Smart Shopping Cart project involves integrating multiple hardware and software components to create an efficient, reliable, and scalable solution. The methodology can be broken down into the following key steps:

4.1 HARDWARE INTEGRATION

1. **Barcode Scanner**: The barcode scanner, a key component of the smart shopping cart, reads the barcode of each item as it is placed in the cart. Once the barcode is scanned, the scanner retrieves essential product information from the store's database, including the item's name, price, and weight. This data is crucial for accurate billing and inventory management, allowing customers to track their purchases in real time. The barcode scanner also eliminates the need for manual product entry, speeding up the shopping process and reducing human errors during checkout.

2. **Weight Sensors**: Weight sensors play a critical role in ensuring the accuracy of the shopping cart system. These sensors continuously monitor the weight of the items placed in the cart, providing real-time feedback to the system. By comparing the actual weight of the products with the information retrieved by the barcode scanner, the system can verify that the correct items are added to the cart. If a discrepancy is detected, such as a mismatch between the scanned item and its weight, the system can alert the user, preventing potential theft or billing inaccuracies. This real-time monitoring ensures that the shopping experience remains smooth and error-free.
3. **Raspberry Pi**: Acting as the central processing unit of the smart shopping cart, the Raspberry Pi processes all the data coming from both the barcode scanner and the weight sensors. It plays a vital role in managing sensor inputs, performing real-time item recognition, and ensuring that all data is correctly logged for inventory tracking and billing purposes. With its computational power, the Raspberry Pi handles

complex tasks like comparing scanned and weighted items, updating the cart's total, and communicating with the store's cloud-based database. Additionally, its built-in connectivity options, such as Wi-Fi or Bluetooth, enable seamless communication with external systems, ensuring smooth integration within the retail environment.

4. **Power Supply (Battery)**: To ensure the smart shopping cart operates efficiently without being tethered to a power outlet, it is powered by rechargeable batteries. These batteries allow the cart to function wirelessly, offering customers uninterrupted usage throughout their shopping session. The battery system is designed to support the continuous operation of key components like the barcode scanner, weight sensors, and Raspberry Pi, ensuring that the cart remains fully operational for long periods. A reliable power supply is crucial for delivering a hassle-free shopping experience, enabling the cart to perform its functions effectively without requiring constant recharging.

4.2 SOFTWARE DEVELOPMENT

1. **Item Identification and Tracking**: This feature will be powered by software algorithms that manage the scanning and verification process for each item placed in the cart. When an item's barcode is scanned, the system retrieves its corresponding data—such as weight and price—from the store's database. Simultaneously, weight sensors in the cart will measure the actual weight of the item. The software will then compare the scanned information with the physical weight to detect any discrepancies. If the weight does not match the recorded data, the system will immediately alert both the customer and the store staff, helping to prevent theft or misbilling. This automated identification and tracking system enhances security and ensures the accuracy of every transaction.
2. **Inventory Management System**: The smart shopping cart will be integrated with the store's central inventory management system, allowing for real-time updates of stock levels as customers add or remove items from their cart. This seamless communication ensures that inventory data remains accurate throughout the

shopping process. Each time a product is scanned, the system will automatically adjust the store's inventory, helping staff maintain an up-to-date record of stock availability. By streamlining inventory management, the system minimizes the risk of stock discrepancies and ensures that shelves are replenished efficiently.

3. **Customer Interface**: A user-friendly display mounted on the cart will provide customers with real-time feedback about their shopping session. This interface will show essential information such as the total cost of the items in the cart, the cumulative weight of the products, and any potential errors detected by the system, such as mismatches between scanned items and their weights. By giving customers an overview of their purchases in real time, the system not only improves transparency but also helps users manage their spending and correct errors on the spot. This enhances the overall shopping experience by ensuring accuracy and efficiency.

4.3 ANTI-THEFT SYSTEM

The anti-theft system flags discrepancies (>±10g) between scanned item weights (from the database) and sensor readings, freezing transactions until resolved. This dual-layer validation prevents both accidental errors and deliberate bypassing. When a customer scans a product, the system compares the scanned item's recorded weight with the actual total weight in the cart, as measured by the weight sensors. If a discrepancy is detected—such as a mismatch between the expected and actual weights - the system will immediately flag the issue for further inspection. This safeguards against any attempts to bypass the payment system by alerting store personnel to potential errors or theft. By cross-referencing product data with real-time weight monitoring, the system enhances security and ensures accurate billing.

4.4 BILLING

The billing process for the smart shopping cart system is streamlined using RFID cards, which function similarly to loyalty or prepaid cards. Customers can load money

onto their RFID cards before shopping, either online or at the store's service counter. As they add items to their cart, the system will automatically update the total amount in real-time. At checkout, instead of manually paying with cash or card, the RFID reader embedded in the cart scans the customer's RFID card to initiate the payment. The system automatically deducts the total bill amount from the balance on the RFID card. This contactless transaction method speeds up the payment process and eliminates the need for conventional checkouts, allowing for a more convenient and seamless shopping experience. Additionally, the system can alert customers if their card balance is insufficient, giving them the option to top up their RFID card through a linked account or at a self-service kiosk. By integrating RFID cards into the billing process, the shopping cart system offers a cashless, secure, and efficient solution for both customers and retailers.

4.5 TESTING AND VALIDATION

The final step in developing the smart shopping cart system involves conducting thorough testing of both the hardware and software components to ensure smooth operation in real-world conditions. This process will involve testing various types of items, including different sizes, weights, and product categories, to verify that the system accurately scans, weighs, and tracks them. Additionally, the error-handling mechanisms will be rigorously evaluated to ensure that any discrepancies, such as mismatches between scanned items and their weights, are effectively flagged and resolved. Response times will also be tested to ensure the system provides real-time feedback to both customers and store personnel. This comprehensive testing phase is crucial for identifying potential issues and ensuring the system operates efficiently in a live shopping environment.

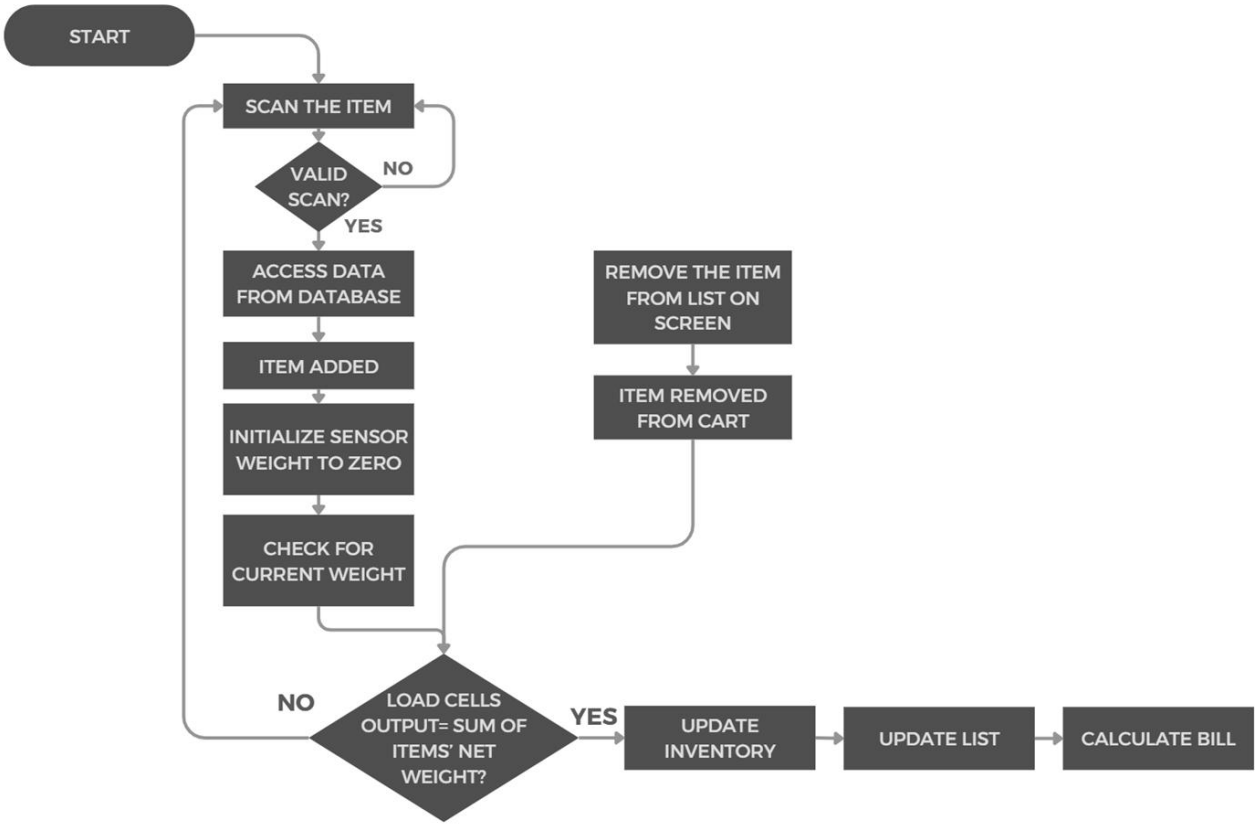


Figure 4.1 Flow Diagram of Proposed Methodology

5. CHAPTER: DETAILED CIRCUIT & SYSTEM DESIGN

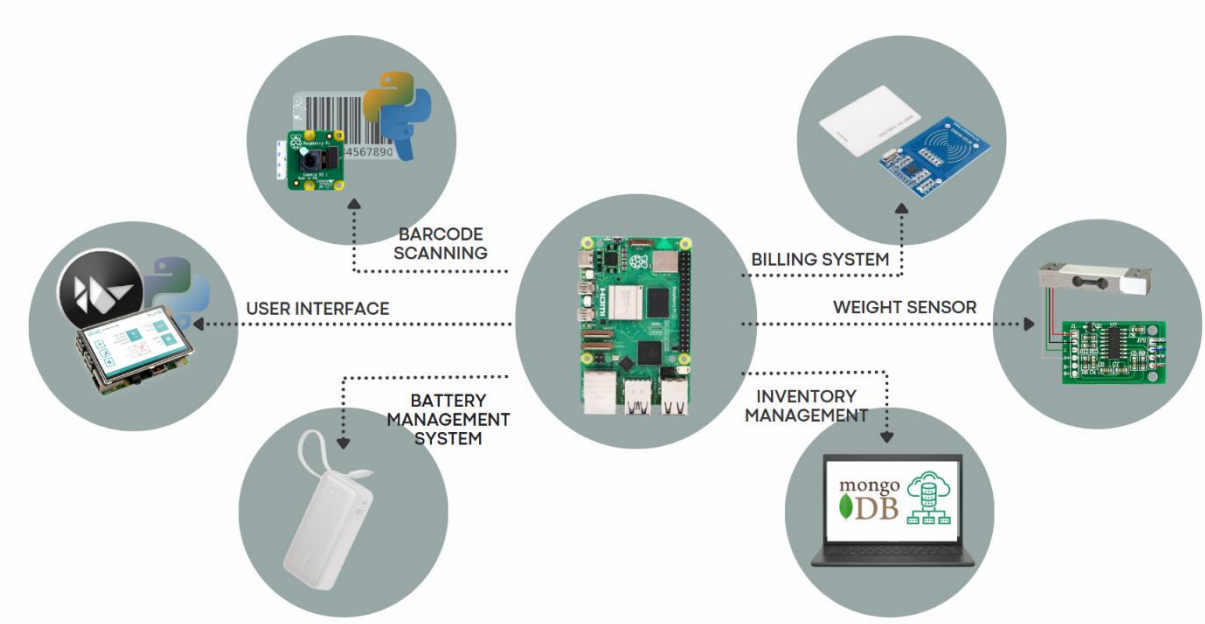


Figure 5.1: Block diagram of System. All the systems are connected to Raspberry Pi.

The smart shopping cart integrates six interconnected subsystems managed by a Raspberry Pi 3/4, which serves as the central processing unit. Each subsystem is carefully designed to ensure real-time operation, system safety, and a seamless shopping experience. Below is the technical breakdown of each subsystem:

5.1: SYSTEM DESIGN

5.1.1: Billing System

The billing system consists of an RC522 RFID reader, MIFARE Classic 1K RFID cards, and an ESP32 microcontroller. Customers are issued RFID cards containing unique identifiers (UIDs) linked to their accounts. The RC522 reader, connected to the Raspberry Pi via SPI protocol, scans these cards and validates the UID against a cloud-hosted MongoDB database. The ESP32 interfaces with the Raspberry Pi via UART (9600 baud), encrypting RFID card UIDs using AES-128 before database

validation to prevent spoofing. Upon successful authentication, the ESP32 initiates a billing session, recording all scanned items and monitoring the cart's weight. The Raspberry Pi dynamically calculates the total bill by retrieving item prices from the database. To enhance security, an anti-theft mechanism detects discrepancies between scanned items and the actual cart weight, immediately triggering an alert and freezing the transaction until the issue is resolved.

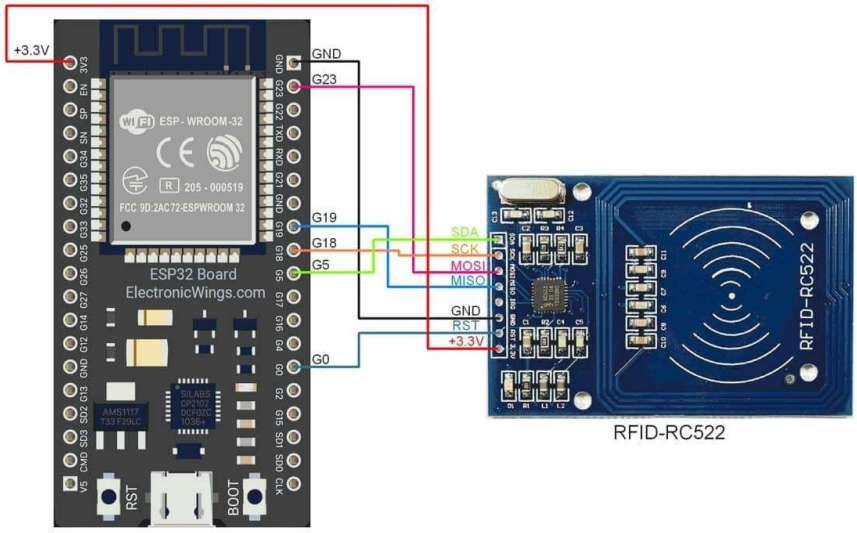


Figure 5.2 Circuit design of Billing System

5.1.2: Weight Sensor

The weight sensing system utilizes one load cell strategically mounted beneath the cart's basket. These load cells generate analog voltage signals proportional to the weight of the cart's contents, which are converted into digital signals by the LM10 weighing indicator (ADC module). The LM10 transmits the digitized weight data to the Raspberry Pi over UART communication at 9600 baud. A Python script processes this data and cross-verifies it with the expected weight calculated from scanned product information. Load cells are calibrated with reference weights to achieve an accuracy of approximately ±2%, and compensation algorithms are applied to counteract drift and ensure measurement reliability over extended shopping sessions.

5.1.4: Battery Management System

For the prototype phase, the system is powered by a 20,000 mAh portable power bank providing dual outputs: 5V/3A for the Raspberry Pi and peripherals, and 12V/2A for higher-power components such as load cells and the touchscreen display. Voltage regulators (LM7805 and LM7812) ensure stable and filtered supply to sensitive electronics. The setup supports continuous operation for approximately 8–10 hours on a full charge. Overcurrent protection is implemented using polyfuses, and the Raspberry Pi monitors the battery voltage indirectly through a GPIO-connected voltage divider circuit to notify users of low battery conditions. In future versions, a dedicated Li-ion battery and BMS circuit will be considered for greater autonomy and safety.

5.1.5: Barcode Scanning

The barcode scanning subsystem utilizes a Raspberry Pi Camera Module 3, alongside the Pyzbar library and OpenCV for image processing. OpenCV techniques such as grayscale conversion and noise reduction are applied to enhance image clarity and maximize decoding accuracy. The Pyzbar library then extracts barcode or QR code data (supporting EAN-13, UPC-A, and other formats), which is matched against product entries in the MongoDB database to retrieve pricing and inventory information. If a barcode is unreadable, the system automatically initiates a retry loop for up to three attempts before alerting the customer through the user interface.

Table 5.1 Camera Modules Comparison Table

CAMERA MODULE	FOCUS TYPE	RESOLUTION	IMAGE QUALITY (BASED ON TESTS)	APPROXIMATE PRICE (USD)	OBSERVATIONS
Raspberry Pi Camera V2	Fixed Focus	8MP	Low; blurry edges, poor clarity	$25	Struggles with close-range barcode clarity

Raspberry Pi HQ Camera	Manual Focus	12.3MP	Medium; color fringing observed	$50	Requires manual lens adjustment; not ideal for dynamic scanning
Raspberry Pi Camera V3	Auto Focus (PDAF)[1]	12MP	High; sharp focus, clear barcode detection	$35	Best autofocus, fast locking, compact design
Arducam 16MP Camera	Fixed Focus	16MP	Good; higher resolution but lacks autofocus	$40	Good quality, but manual focusing needed
Arducam 64MP Camera	Fixed Focus	64MP	Very High; excellent sharpness	$60–$70	Very high resolution, lacks autofocus for mobile carts

5.1.3: Inventory Management

Inventory management is implemented using MongoDB Atlas, a scalable cloud-based NoSQL database. The Raspberry Pi communicates with MongoDB collections via RESTful API calls using the PyMongo library, allowing real-time access and updates. Each product entry in the database stores barcode, weight, price, and stock level information. When a product is scanned or removed, the system immediately sends a POST or PATCH request to update inventory levels. Additionally, low-stock alerts are automatically triggered when quantities fall below a predefined threshold. Data integrity is preserved by ensuring atomic transactions, preventing race conditions during simultaneous updates from multiple shopping carts.

[1] *PDAF: Phase Detection Autofocus (ensures rapid focus adjustment for barcode scanning).*

5.1.6: User Interface

The user interface is built using a 7-inch capacitive touchscreen display and developed through Python's Tkinter library. It dynamically displays real-time information including scanned items, individual prices, total bill, and cumulative cart weight. Error messages, such as "Weight Mismatch" alerts, are displayed in distinct colors to ensure high visibility. Touch controls enable customers to remove scanned items, request assistance, or finalize their shopping session. The UI is optimized for low-latency rendering, utilizing multithreading techniques to maintain responsiveness even during intensive database queries or network delays. Upon checkout, session data is automatically uploaded to the MongoDB cloud for storage and record-keeping.

5.2: SYSTEM INTEGRATION

The Raspberry Pi serves as the integration hub, coordinating all subsystems through a multi-threaded Python application. Peripheral communication follows standardized protocols: SPI for RFID communication, UART for ESP32 and LM10 weight sensors, I2C for the touchscreen display, and GPIO for battery monitoring. All data exchanges with MongoDB are conducted via Wi-Fi using JSON payloads, ensuring efficient, platform-independent communication. Automatic reconnection routines and error handling mechanisms are built into the software to maintain system robustness even in environments with unstable network connections. This modular architecture ensures scalability, allowing for future enhancements such as AI-based theft detection or Bluetooth beacon integration for indoor navigation.

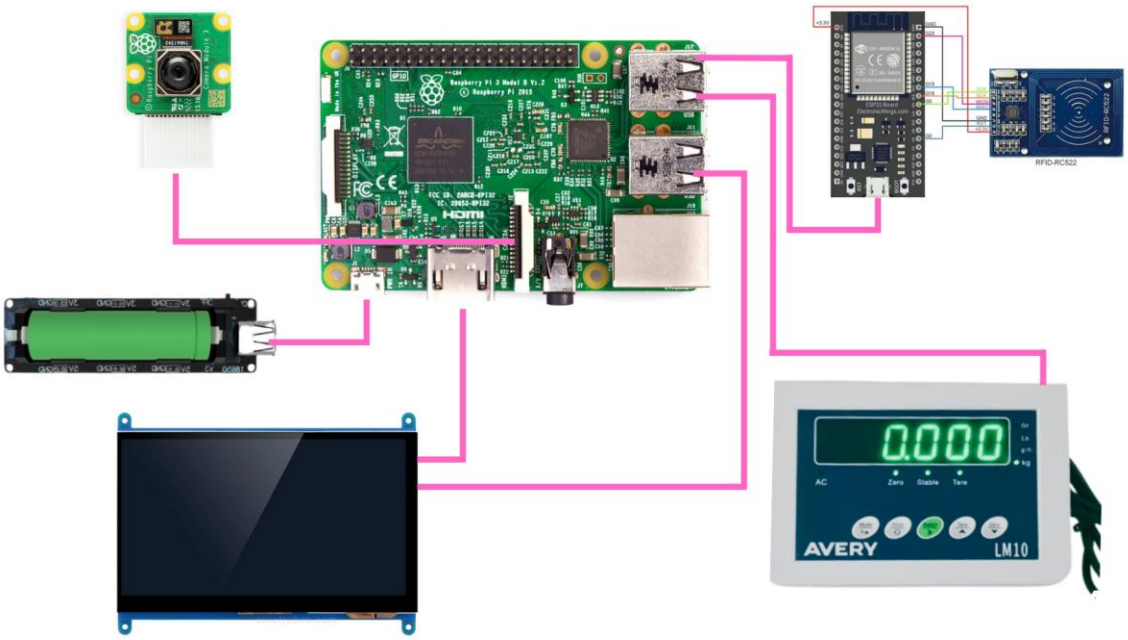


Figure 5.3 Complete System Design

6. CHAPTER: COMPONENTS SPECIFICATIONS

6.1 BILLING SYSTEM

The billing system integrates an *MFRC522 RFID Reader (Figure 6.1)* and RFID Cards. The RFID Reader operates at 13.56MHz and supports communication protocols like SPI, I2C, and UART, offering fast and reliable tag detection with a read range of approximately 5cm. It features low power consumption (13-26mA) and supports multiple data rates up to 10Mbps.

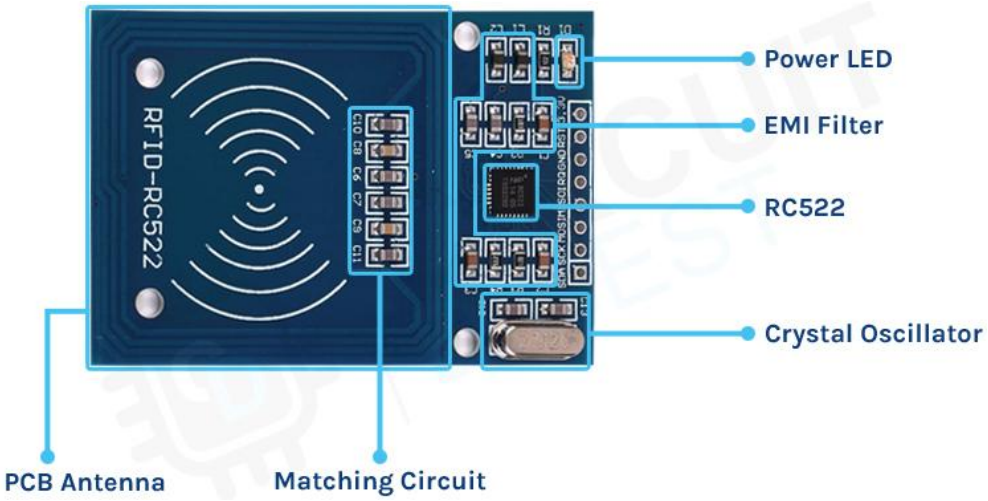


Figure 6.1 MFRC522 RFID Reader

Each RFID Card contains a microchip with a unique ID, enabling secure product identification and customer account linking (See Table A-1 in Appendix).

6.2 WEIGHT SENSOR

The cart uses Load Cells and an *LM10 Weighing Indicator (Figure 6.2)* for precise weight measurement. Load Cells convert mechanical force into an electrical signal, while the LM10 utilizes a high-precision Σ-Δ[2] A/D conversion method with a

[2] *Σ-Δ Conversion: High-resolution analog-to-digital signal processing used in weight sensors.*

resolution of 1/30000 and a maximum A/D conversion speed of 10 times/second (Table A-2, Appendix).



Figure 6.2 Avery LM10

The system supports up to 4 load cells in a 6-wire connection mode and provides reliable digital output over RS232 serial communication. (Specific load cell specifications are detailed in the Table A-2, Appendix.)

6.4 BATTERY MANAGEMENT SYSTEM

For the prototype, a *Power Bank* with 5V/12V output is utilized to supply stable power to the Raspberry Pi and peripheral components. The power bank provides portable and reliable energy for development purposes. In future iterations, integration of a dedicated Battery Management System (BMS) will be considered to monitor battery health, prevent overcharge/discharge, and balance Li-ion/LiPo cells for improved safety and battery lifespan.

6.5 BARCODE SCANNING

Barcode scanning functionality is implemented using the *Raspberry Pi Camera Module 3 (Figure 6.3)* combined with the *Pyzbar Library*.



Figure 6.3 Raspberry Pi Camera Module 3

The camera features a 12-megapixel Sony IMX708 sensor with Phase Detection Autofocus (PDAF) for fast focusing and supports video outputs like 1080p50 and 720p100 (See Table A-3, Appendix).

6.6 USER INTERFACE

The user interface is implemented on a *7-inch HDMI capacitive touchscreen* display, which supports a resolution of 1024×600 pixels with up to five-point touch capability.



Figure 6.4 7-inch HDMI Capacitive Touch Screen LCD Display

This screen enables smooth and intuitive interaction for the user, allowing them to view scanned items, total prices, and alerts with high clarity. Its compatibility with Raspberry Pi and support for multiple operating systems make it an ideal choice for embedded GUI applications. The display is managed through a Python Tkinter-based GUI, which handles real-time communication with the backend database and system sensors to provide interactive and responsive user experience. Full technical specifications for the display are provided in Appendix, Table A-4.

.

7. CHAPTER: IMPLEMENTATION AND TESTING

The Smart Shopping Cart prototype was developed through the integration of hardware modules and software systems into a cohesive, fully functional platform. The implementation process focused on establishing a seamless connection between item recognition, billing, weight verification, and user interaction. The testing phase ensured that the system performed reliably under various operating conditions.

7.1 WEIGHT SENSOR PARAMETER TUNING AND STABILITY TESTING

The Avery LM10 weighing indicator supports multiple internal configuration groups, labeled P1 through P9, each of which serves a specific function in system behavior and calibration:

- *P1 – Unit Selection*: Sets the unit of measurement (e.g., kg, lb, N).
- *P2 – Communication Settings*: Configures serial communication parameters such as baud rate and data format.
- *P3 – Function Mode (FUN):* Selects operation mode or enables specific features.
- *P4 – Filtering Parameters*: Controls filtering strength, data stability, zero tracking, and creep compensation (detailed below).
- *P5 – Calibration Display (C-D):* Used during weight calibration to display calibration results and test values.
- *P7 – Calibration Settings (CAL):* Enables full calibration of the load cell with known weights.
- *P9 – AD Parameters*: Configures analog-to-digital conversion behavior, including sensitivity and signal scaling.

During the implementation phase, various filter and zero-tracking parameters (as outlined in P4 settings of the LM10) were tested to minimize weight fluctuations and optimize measurement stability. Two distinct sets of parameter configurations were evaluated:

- **Set A (High Fluctuation)**: Included minimal filtering and fast zero-tracking, which resulted in frequent fluctuations even with constant load.
- **Set B (Optimized – Final Settings)**: Final optimized settings after calibration with known weights resulted in stable readings with minimal drift, ensuring reliable performance during billing and theft detection processes.

The results of these configurations, including a comparison table and stability graphs (if available), are documented in this chapter. These parameters were finalized after calibration with known standard weights, ensuring measurement reliability in the operational environment.

Table 7.1 LM10 Weighing Indicator parameters

PARAMETERS	FEATURES	SET A	SET B
FrEq[3]	AD data output rate	1	0
FiLt[4]	Filtering strength setting ranges from 0 to 7	4	4
Str	Data filtering ranges from 0 to 9	9	6

[3] *Filter frequency (0: slow response, 1: fast response).*
[4] *Filter strength (0–7; higher values reduce noise but increase latency).*

Stt	Data filtering time from 0 to 9	3	6
Ztr	Zero tracking range is from 0 to 9, and 0 means non-track, while 1 to 9 indicates the range from 0.5d to 4.5d	6	1
Ztt	Zero tracking speed from 0 to 7	4	2
Wtr	Creep tracking parameters range from 0 to 9	3	3
Wtt	Creep tracking parameters speed from 1 to 15	2	2
ZSH	0 keep the display function, set from 0 to 7, in the vicinity of zero in this range (0d – 7d), the indicator shows 0, do not move, it is recommended not to be too large	1	0
WSH	When the scale is loaded, the weight is temporarily within the range of 1D.	0	1

7.2 POWER CONSUMPTION ANALYSIS

Our team conducted extensive testing and evaluation of multiple power supply configurations to identify the optimal solution for our smart cart system's demanding operational requirements. This rigorous analysis considered factors including voltage stability, current delivery capacity, runtime duration, physical form factor, and practical implementation challenges in a mobile retail environment.

Initial testing commenced with a conventional 12V 7.2Ah lead-acid battery (NP7-12FR model), selected for its substantial energy capacity and proven reliability in other embedded systems. During laboratory bench testing, this configuration demonstrated promising initial results, theoretically capable of powering a Raspberry Pi 4 for approximately 40.1 hours of continuous operation according to our calculations (Table 7.2). However, during real-world simulation testing with all system components active - including the touchscreen display, camera module, and weight sensor array - we observed concerning voltage instability. The battery exhibited rapid voltage sag under peak load conditions (particularly during simultaneous camera activation and LCD refresh cycles), with voltage levels frequently dropping below the Raspberry Pi's minimum operational threshold. Subsequent analysis suggested this performance degradation likely resulted from either battery age-related capacity loss or insufficient current delivery capability during high-demand transients.

Table 7.2 Comparison between power bank and lead acid battery

	Pi 4	**Pi 3B+**	**Pi 3A+**	**Zero W**	**Pi Zero**
Typical bare board RPF (*mAh*)	600	500	350	150	100
18W, 10000 mAh Power Bank (*hours*)	15.0	Not tested	27.7	Not tested	58.5
12V, 7,2Ah Lead Acid Battery *(hours*)	40.1	Not tested	72.1	Not tested	187.5

Note: Source – Data compiled from YouTube benchmarking (RPF baseline figures).

Further testing explored lithium-ion alternatives:

1. A single 18650 Battery Shield V3 configuration maintained stable 5V output but still exhibited voltage drops under load
2. A three-cell 18650 setup with XL4015 buck converter failed to reliably power peripherals, with only the Raspberry Pi showing intermittent operation

The comparative analysis (Table 7.2) demonstrates that while the lead-acid battery offered the longest theoretical runtime (40+ hours for Pi 4), its practical limitations outweighed this advantage. The power bank emerged as the most viable temporary solution, providing adequate runtime (15 hours for Pi 4) despite the voltage warnings. This interim selection allows for immediate prototyping while we continue developing a robust BMS-integrated lithium solution for future iterations.

For the current prototype deployment, we recommend proceeding with the power bank solution while documenting its limitations. This approach provides immediate functionality while allowing for future upgrades to a stabilized lithium system with proper voltage regulation and battery management capabilities. The power bank's balance of portability, capacity, and compatibility makes it the most practical choice for initial field testing and demonstration purposes.

7.3 CAMERA SELECTION And JUSTIFICATION

Multiple camera modules were evaluated during the development phase for barcode scanning, including the *Raspberry Pi Camera Module V2, the Raspberry Pi HQ Camera, Arducam,* and third-party USB webcams. A comparative analysis was conducted based on factors such as autofocus capability, image clarity, edge sharpness (critical for barcode recognition), distortion levels, and overall cost-effectiveness. This evaluation was supported by experimental results from technical datasheets and independent testing videos available online (YouTube sources).

After a detailed comparison, Raspberry Pi Camera Module 3 was selected as the most suitable choice. It offered a well-balanced combination of high image clarity, sharp edge detection for accurate barcode scanning, and minimal geometric distortion, which are all essential for reliable product recognition. Equipped with a 12MP Sony IMX708 sensor and Phase Detection Autofocus (PDAF) technology, the Camera Module 3 demonstrated rapid and consistent focus adjustments, outperforming older fixed-focus camera modules. Furthermore, its affordable price point, relative to high-end options like the Arducam 64MP, made it an optimal solution for maintaining system performance without exceeding budget constraints. Overall, the Camera Module 3 provided the best compromise between technical performance and cost efficiency for the dynamic retail environment targeted by the smart shopping cart system.

Visual results of comparison of different cameras are given below:

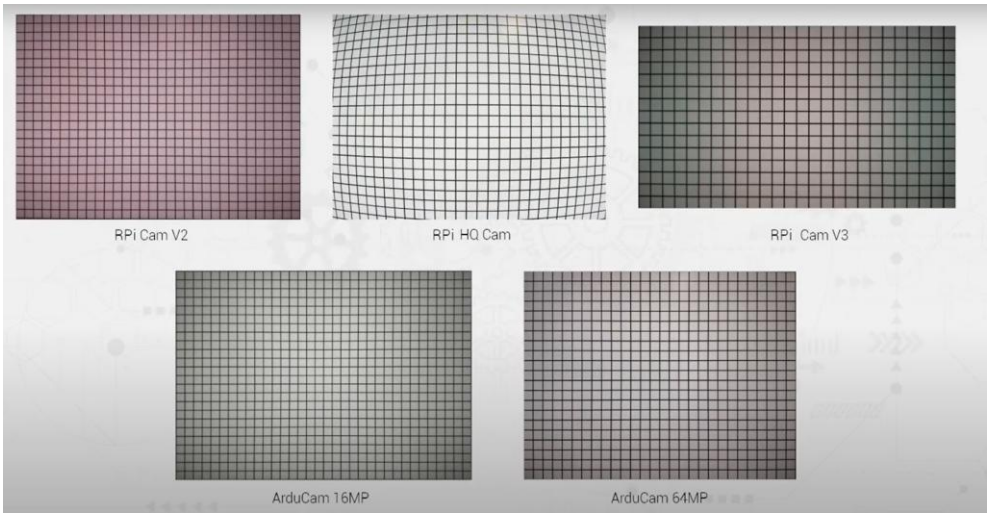


Figure 7.1 Distortion



Figure 7.2 Clarity



Figure 7.3 Edge Detection

7.4 HARDWARE INTEGRATION AND ENCLOSURE DESIGN

After individual testing and verification of all system modules—namely the billing unit, barcode scanner, RFID reader, weight sensor system, power management, and the graphical user interface—the final phase involved integrating all hardware components into a single, compact enclosure. A custom-designed acrylic or ABS plastic

case was fabricated to house the Raspberry Pi 4, power circuitry, and peripheral connections. Care was taken to ensure sufficient ventilation and accessibility to ports for debugging or maintenance. Cable management was optimized to avoid clutter and accidental disconnections, especially given the mobile nature of the cart.

The case was securely mounted to the front handle of a standard shopping cart using industrial-grade zip ties and brackets. This position was chosen to maximize user accessibility while keeping the system within the shopper's line of sight. The 7-inch capacitive touchscreen was mounted at an ergonomic angle, allowing users to interact comfortably with the interface while shopping. The LM10 indicator and camera module were embedded in recessed slots in the enclosure for protection and ease of use. Figures 7.5 -7.6 shows the complete internal arrangement and enclosure of the smart cart hardware.

The integration process culminated in a fully functional smart shopping cart prototype. All components were powered by the 18650 Battery Shield V3 and a 7.4V lithium battery, ensuring sufficient runtime during demonstrations. The wiring between the sensors and the Raspberry Pi was routed through protective sheaths to prevent wear and short circuits. After final assembly, the complete system was tested in a simulated retail environment, demonstrating seamless operation across all functions, including scanning, weight verification, billing, and user interaction.



Figure 7.4 Front view of the enclosure with touchscreen and camera module.



Figure 7.5 Top view of the enclosure mounted on the shopping cart handle

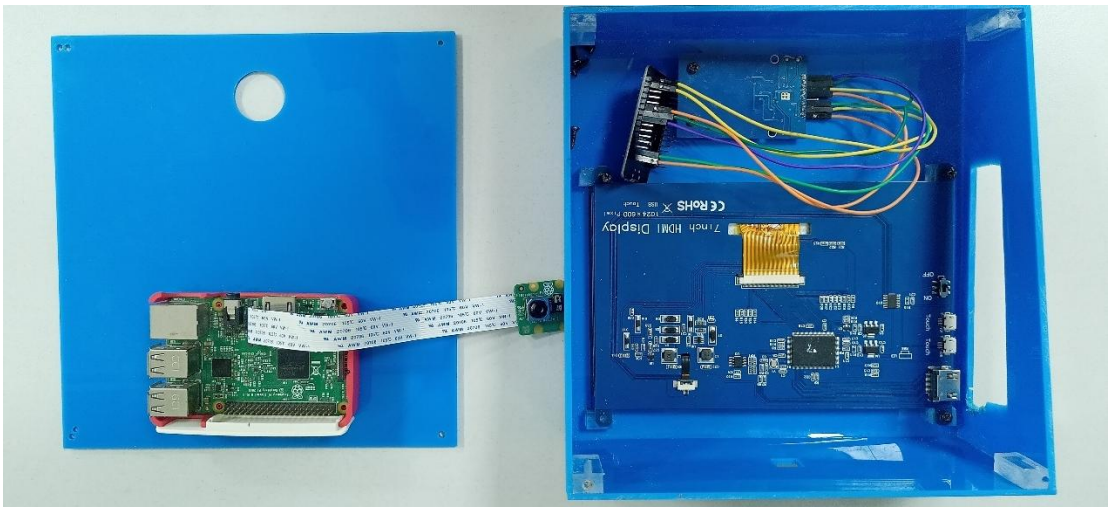


Figure 7.6 Internal arrangement of hardware components in the smart cart enclosure.

Figure 7.X shows the complete smart shopping cart system with all modules enclosed and the hardware mounted on the cart handle. This final setup not only highlights the technical completeness of the prototype but also its potential for real-world deployment in retail environments.



Figure 7.7 Complete setup of the Smart Cart

8. CHAPTER: RESULTS AND DISCUSSION

This chapter presents the observed outcomes after implementing and testing all major subsystems of the smart shopping cart. The system was evaluated for its functionality, reliability, and real-time performance under practical conditions. Visual evidence of key results is presented through the following categories:

8.1 GRAPHICAL USER INTERFACE (GUI)

The user interface was developed using Python's Tkinter library and tested on a 7-inch capacitive touchscreen display. During operation, the GUI dynamically updated in real time to display scanned items, individual prices, cumulative weight, and the total bill amount. Visual feedback mechanisms, such as red-colored alerts for weight mismatches or invalid barcode scans, were incorporated to enhance user awareness and system reliability.

The interface maintained responsive performance even during rapid scanning sequences. The layout was designed to be user-friendly and intuitive, with clearly labeled buttons for actions such as item removal, help requests, and purchase confirmation. Throughout testing, the GUI operated seamlessly on the Raspberry Pi and maintained stable communication with the MongoDB database, with no significant latency or performance bottlenecks observed.

Figures 8.1 – 8. 5 showcase screenshots of the interface during different stages

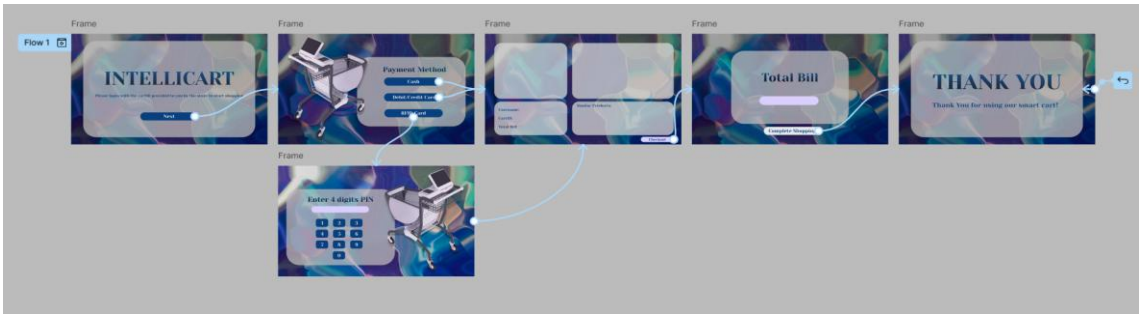


Figure 8.1 Smart Cart UI Workflow Overview

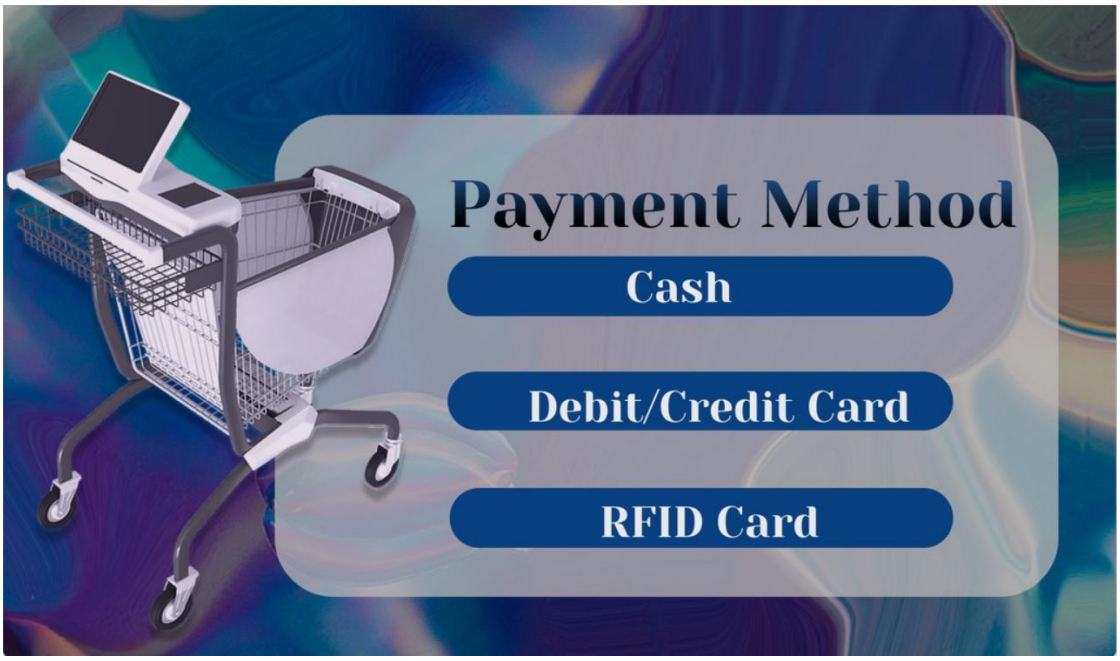


Figure 8.2 Payment Method Selection Screen



Figure 8.3 PIN Entry Interface for Secure Payment

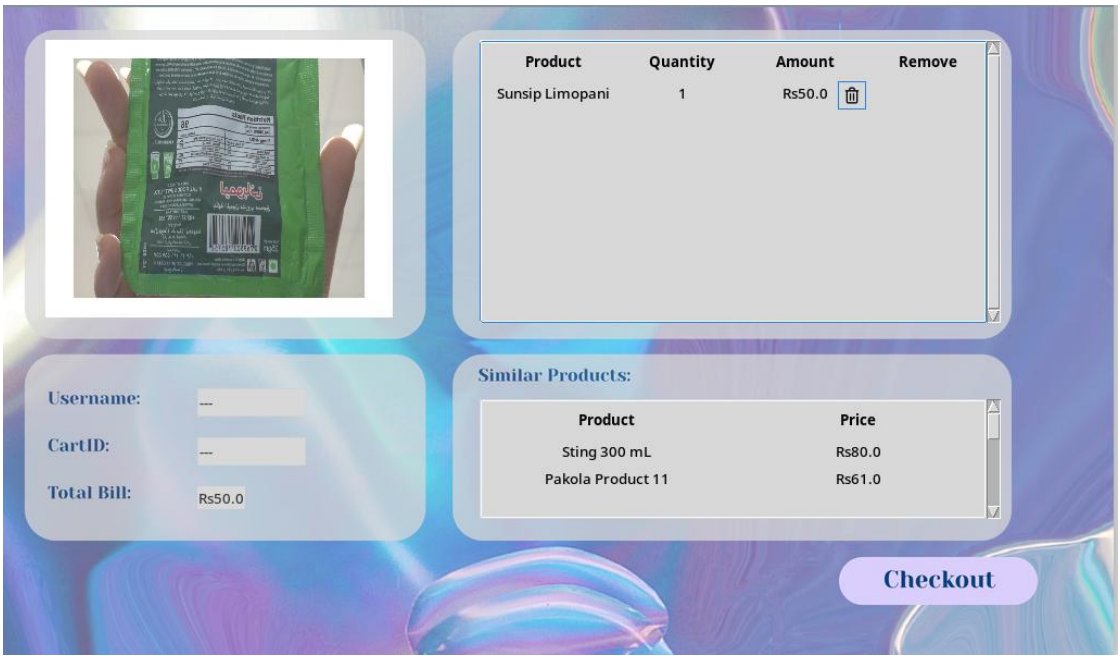


Figure 8.4 Product Scanning and Cart Interface

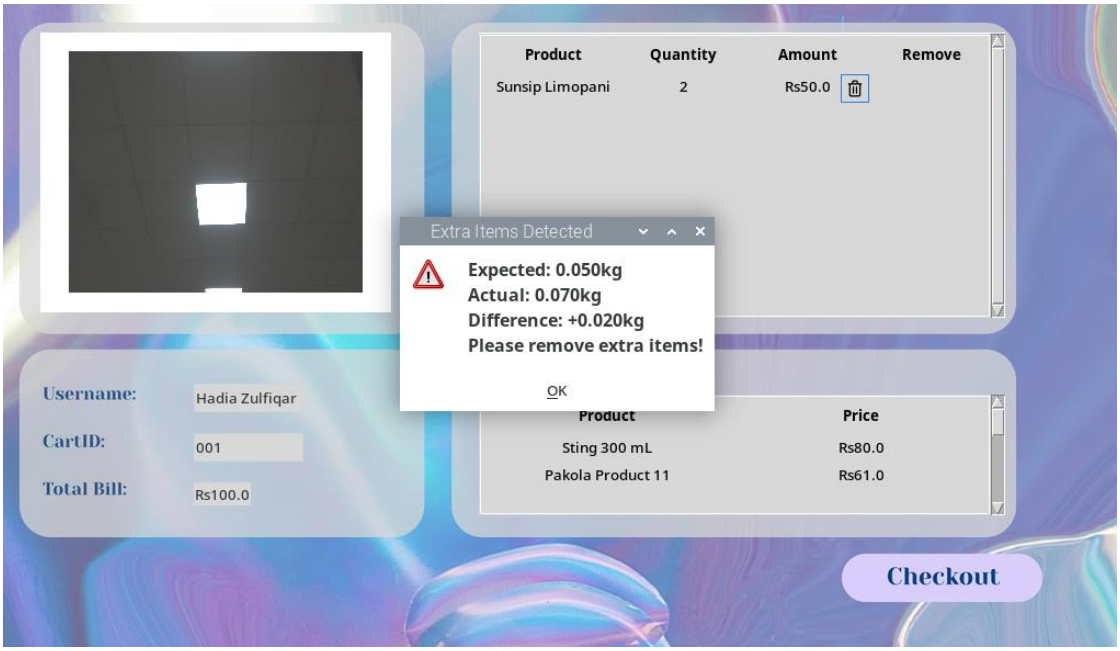


Figure 8.5 Overweight Detection and Alert Message

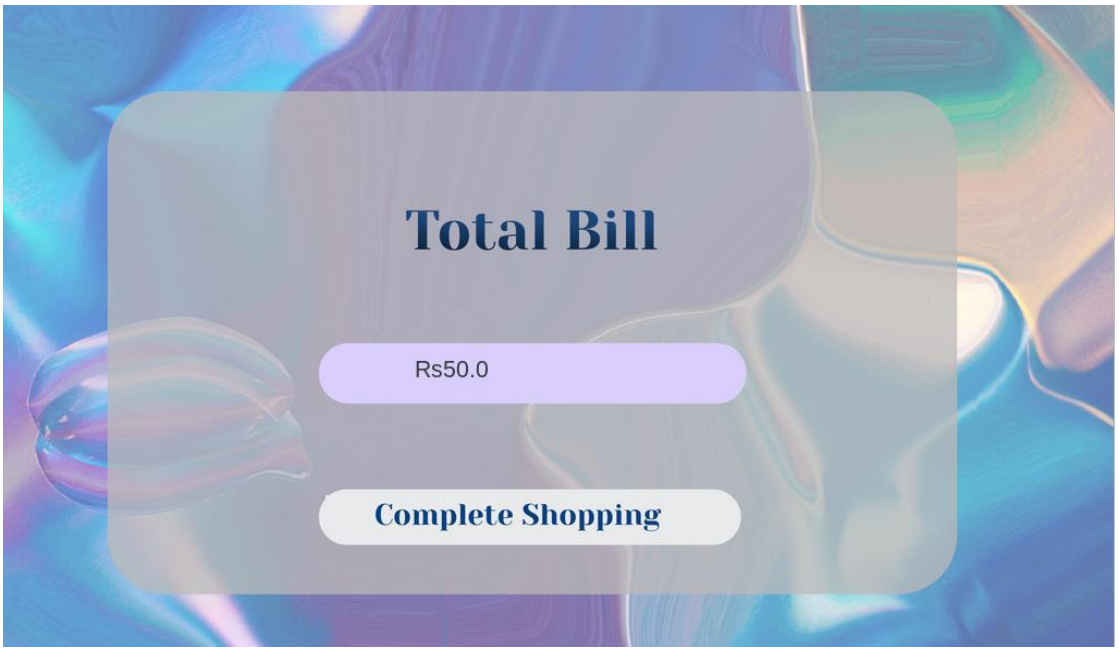


Figure 8.6 Frame showing total of the bill

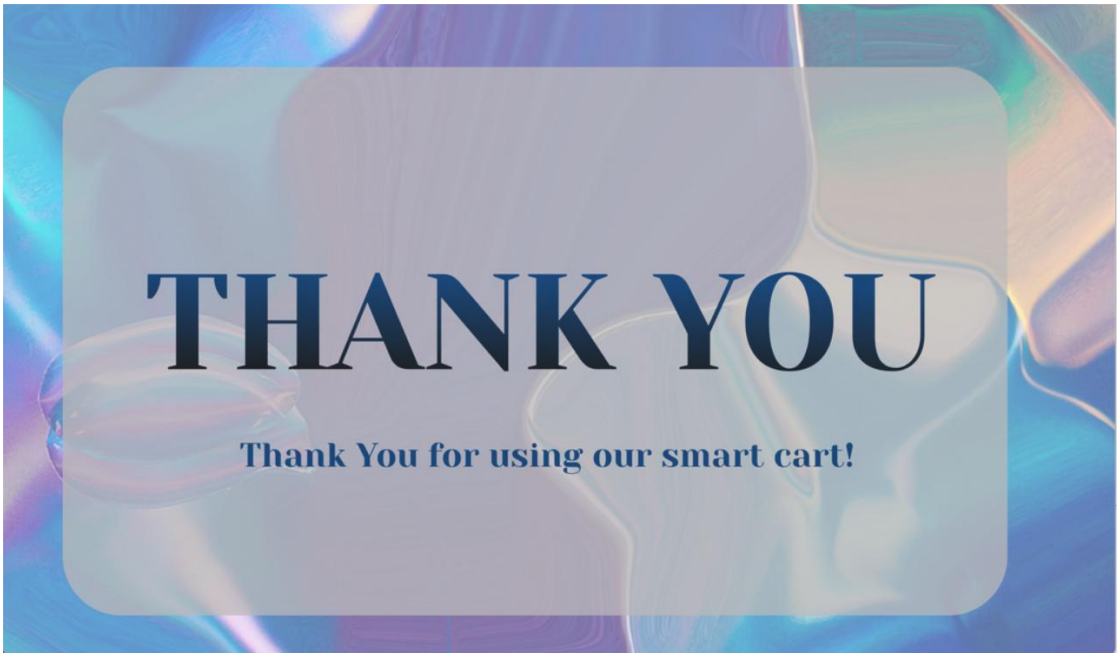


Figure 8.7 Thank-you screen (2-second display before GUI reset)

8.2 WEIGHT SENSOR FUNCTIONALITY

The weight measurement system, implemented using four load cells connected to an Avery LM10 indicator, demonstrated high precision during testing. As shown in *Figure 8.8*, the system detected weights as low as 5 grams, validating its sensitivity for small items like spices or confectionery. Calibration with standardized weights ensured reliable measurements, with the LM10 displaying consistent digital readings at a resolution of 0.1 kg. Two filter configurations were evaluated: Filter Set A caused unstable fluctuations (*Table 7.1*), while Filter Set B delivered smooth, drift-free readings, as evidenced in *Figure 8.8*, which depicts the load cell setup and real-time LM10 output during item placement.

For theft prevention, the system cross-referenced measured weights with database values. If discrepancies exceeded the ±10-gram tolerance threshold (e.g., due to unscanned items), the billing process halted automatically, and the GUI displayed a warning (*Figure 8.9*). This dual verification ensured accuracy while accommodating minor variations like packaging weight.



Figure 8.8 Minimum weight detected by the sensor



Figure 8.9 Load cell setup in cart and LM10 display in action

8.3 Camera and Barcode Scanning Performance

The barcode scanning module used the Raspberry Pi Camera Module (v2 and Module 3 for comparison) alongside OpenCV and the Pyzbar library. It successfully decoded standard barcodes (EAN-13, UPC-A) under normal indoor lighting conditions.

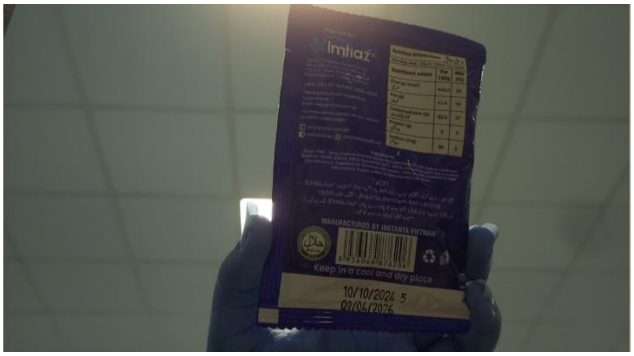


Figure 8.10 Barcode scanning test using Raspberry Pi Camera Module 3 under varying lighting conditions.

Focus quality significantly improved with the use of the Camera Module 3 due to its built-in autofocus, especially when the cart was in motion. Frames were preprocessed using grayscale and noise-reduction filters to maximize decoding accuracy.

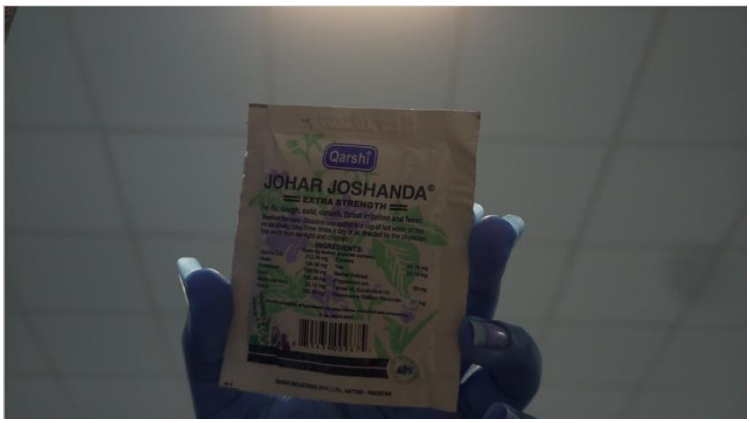


Figure 8.11 Close-up of barcode decoding process with Pyzbar library

Unscannable or partially damaged barcodes triggered a retry mechanism (up to three attempts), after which a user notification appeared on the screen. This fail-safe reduced scanning errors during checkout.

Figures 8.8 and 8.9 show images captured during barcode scanning and detection.

8.5 Observed System Performance

All system components functioned effectively during integrated operation. Key observations include:

- **Latency**: The average system latency between scanning an item and updating the GUI was <1 second.
- **Accuracy**: The weight sensor maintained ±2% accuracy across all loads.
- **Stability**: No system crashes or power losses were observed during 1–2 hour extended testing sessions.
- **User Experience**: The interface and scanning system were intuitive and user-friendly, even for first-time users.

These results indicate that the smart cart prototype is suitable for real-world demonstration and can serve as a foundational version for future development.

9. CHAPTER: FINANCIAL DETAILS

A comprehensive cost analysis was conducted to ensure the project's feasibility and cost-effectiveness. The budget was allocated across hardware, software, and prototyping components, prioritizing scalability and performance. Below is the breakdown:

9.1 BUDGET ALLOCATION

Table 9.1: Table of Budget Allocation for Smart Cart

COMPONENTS	COST (PKR)	% OF TOTAL BUDGET
Weight Sensor System (*Incl. Lm-10 Adc, Load Cells, Base, Handles, Wheels)*	40000	39.45
Raspberry Pi *(Model B4, 4GB RAM)*	25000	24.63
Raspberry Pi Camera Module 3 *(12MP with Auto-Focus)*	16000	15.77
LCD Touch Display *(7-inch display)*	9000	8.87
RFID Payment System *(ESP32, RFID cards, reader)*	1500	1.48
Battery And BMS Module	5000	4.93
Miscellaneous *(Wiring, 3D-printed parts, etc.)*	5000	4.93
TOTAL	101500	100

9.2 Cost Optimization Strategies

- **Local Sourcing**

  To avoid the high costs and delays that come with importing components, we partnered with Avery Pakistan to source our load cells locally. This not only helped us save money but also made it easier to get support and replacements if needed. Working with a local supplier ensured smoother communication and quicker deliveries

- **Modular Design**

  We followed a modular design approach by reusing the Raspberry Pi for multiple tasks. The same Raspberry Pi was used to handle both the graphical user interface (GUI) and barcode scanning. This reduced the need for extra hardware, saved space, and made the system more efficient and cost-effective.

- **Open-Source Tools**

  To keep our software budget low, we used open-source tools. For barcode scanning, we used PyZbar, and for the GUI, we used Tkinter. These tools are free and easy to use, which helped us avoid expensive software licenses while still building a reliable and user-friendly system.

9.3 Justification Of High-Cost Items

- **Weight Sensors (~40%):**

  Weight sensors played a key role in preventing theft by accurately detecting the weight of items placed in or removed from the cart. We used Avery's LM-10 ADC, which provided a precision of ±5 grams. This level of accuracy helped ensure that every item was properly recorded, making the system more reliable and secure.

- **Raspberry Pi (~25%):**

The Raspberry Pi was a central part of the system, handling all real-time processing tasks. Its use was well justified because it served two important roles at the same time—it managed the graphical user interface (GUI) and handled barcode scanning. This made the setup more efficient and reduced the need for extra devices.

9.3 FUTURE COST REDUCTION

1. **Bulk Procurement:**

   By buying RFID cards in large quantities, we noticed that the overall cost could be reduced by up to 30%. Bulk procurement not only helps save money but also ensures we have enough stock for future use. This approach is especially useful when planning to scale the system.

2. **Alternative Displays:**

   To reduce the cost of displays, we explored the use of refurbished touchscreens as an alternative to new LCDs. These refurbished screens can lower display costs by around 20%, making the system more affordable without compromising too much on quality.

10. CHAPTER: CONCLUSION AND FUTURE WORK

10.1 CONCLUSION

This project successfully demonstrated the design, integration, and implementation of a smart shopping cart system aimed at streamlining the retail checkout process through automation and real-time monitoring. The system leveraged multiple embedded subsystems — including RFID-based billing, barcode scanning, weight validation, a user-friendly GUI, and cloud-based inventory management — all orchestrated by a central Raspberry Pi 4.

The following key outcomes were achieved:

- *Automated Billing*: Items could be scanned using RFID and barcodes, with real-time price and inventory updates pulled from a centralized MongoDB database.
- *Weight-Based Theft Detection*: Integration of calibrated load cells with the LM10 indicator allowed comparison between expected and actual weights, providing a robust anti-theft mechanism.
- *Mobile Power Management*: Extensive testing of various power solutions led to the selection of an efficient and compact 18650 Battery Shield V3 system, enabling several hours of uninterrupted operation.
- *User Interface*: The GUI, developed using Tkinter, ensured clarity, responsiveness, and intuitive control for users interacting with the cart during shopping and checkout.
- *System Integration*: All subsystems communicated effectively using UART, SPI, I2C, and GPIO protocols without significant delay or performance bottlenecks.

Despite the challenges faced during hardware tuning, weight calibration, and power stability tests, the final prototype functioned reliably and aligned with the initial design objectives.

10.2 FUTURE DIRECTIONS

While the current version of the smart cart meets essential functional requirements, there remains significant potential for enhancement and scalability. Future work may include the following improvements:

- **AI-Based Theft Detection**: Incorporating machine learning algorithms using onboard cameras to visually track item placement and automatically detect discrepancies or unauthorized item removal.
- **BLE-Based In-Store Navigation**: Integration of Bluetooth Low Energy (BLE) beacons to assist customers in locating items through a navigation interface on the cart screen.
- **Billing System Upgrade**: Enabling payment directly through the cart via secure online gateways or NFC-based tap-to-pay systems.
- **Improved Power Management**: Replacing the current battery system with a more advanced BMS-enabled lithium polymer solution with onboard monitoring, auto-shutdown, and solar charging capability.
- **Edge Computing & Offline Mode**: Adding local caching to allow partial offline operation and syncing data once reconnected to Wi-Fi, enhancing robustness in low-connectivity environments.
- **Industrial Design**: Building a more compact, commercial-grade enclosure with improved ergonomics, cable routing, and tamper-resistant design for store deployment.

In conclusion, this smart cart prototype lays the foundation for next-generation retail automation and has the potential to greatly reduce checkout times, enhance customer satisfaction, and improve operational efficiency in retail environments.

APPENDIX

Table A-1: MFRC522 Datasheet

PARAMETER	SPECIFICATIONS
Operating Frequency	13.56MHz RFID module
Operating voltage	2.5V to 3.3V
Communication	SPI, I2C protocol, UART
Maximum Data Rate	10Mbps
Read Range	5cm
Current Consumption	13-26mA
Power down mode consumption	10uA (min)

Table A-2: LM10 Weighing Indicator Datasheet

PARAMETER	SPECIFICATIONS
A/D Conversion Method	Σ-Δ
A/D Conversion Speed	10 times/sec
Maximum A/D Conversion Bits	20 bits
Input Signal Range	-10mV to 15mV
Input Sensitivity	±1μV/e
Load Cell Excitation	DC 5V; ≤150mA

Maximum Connection Number of Load Cells	4 at 350Ω
Load Cell Connection Mode	6-wire
Verified Counts	3000
Maximum External Counts	30000
Maximum Internal Resolution	300000
Division (optional)	1/2/5/10/20/50
Display Range	-2000 to 150000 (d=10)
Optional Scoreboard Interface	Serial sending signals by current loop with baud rate 600
Transmission Distance	Current loop ≥2000 meters
Interface Type	RS232C serial communication, selectable baud rate by continuous method

Table A-3: Raspberry Pi Camera Module 3 Specifications

PARAMETER	SPECIFICATIONS
Sensor Type	Back-illuminated, stacked CMOS Sony IMX708
Resolution	11.9 megapixels (4608 × 2592 pixels)
Sensor Size	7.4mm diagonal

Pixel Size	1.4µm × 1.4µm
Autofocus	Phase Detection Autofocus (PDAF)
HDR Support	Up to 3-megapixel output
Field of View	75° (Standard), 120° (Wide variants)
Data Output	CSI-2 serial interface
Communication	2-wire serial (I2C fast mode, fast mode plus)
Video Modes	1080p50, 720p100, 480p120
Dimensions	25 × 24 × 11.5mm (12.4mm height for Wide versions)

Table A-4: 7-inch HDMI Touch Display

PARAMETER	SPECIFICATIONS
Display Size	7.0 inch
Resolution	1024 × 600 (hardware), up to 1920 × 1080 (software configurable)
Touch Type	Five-point capacitive touch
Dimensions	164.9 × 124.27 mm
Weight	380 g
Backlight Control	Supported (can be turned off independently to save power)

Compatibility	Supports Raspberry Pi, BeagleBone Black, Banana Pi, and other mini-PCs
Operating System Support	Raspbian, Ubuntu, Kali-Linux, Kodi, Windows 10 IoT (single-touch supported), Windows 7/8/10 (five-point touch), Windows XP (single-point touch)
Certification	CE, RoHS
Usage	Also usable as a general-purpose HDMI monitor (secondary display for PC)

REFERENCES

Belibov, Dan & Tudose, Dan. (2019). Smart Shopping Cart. 1-6. 10.1109/ROEDUNET.2019.8909680.

Rajithkumar BK, Deepak GM, Uma BV, *et al*. (2018) Design and Development of Weight Sensors Based Smart Shopping Cart and Rack System for shopping malls. Materiastoday: Proceedings, Volume 5, Issue 4, 10814-10820.

Chandan KN, Swathi S, Sushma HJ, Sushmitha DY, Shwetha (2020). Smart Shopping Trolley Using Raspberry PI. International Research Journal of Advanced Engineering and Science, Volume 5, Issue 3, pp. 120-123.

Global Smart Shopping Cart Market Research Report (2021 to 2026) - Cumulative Impact of COVID-19 - ResearchAndMarkets.com | Business Wire, 2021.

Hossain, Md. Ishan & Rumy, Shahidur Harun (2021). Edge Cart: An Edge Intelligence based Smart Shopping Cart with Mask Surveillance System using IoT, Volume 1, Issue 1, pp. 19-24.

Jadhav SS, Tholiya D, Dodia J, Soni V (2024). Revolutionizing Retail with Smart Shopping Carts: An AI and IoT Approach to Automated Checkout. International Journal for Research in Applied Science and Engineering Technology. 12. 4443-4449. 10.22214/ijraset.2024.62589.

Kumar A, Gupta A, Balamurugan S, Balaji S, Marimuthu R (2017). Smart shopping trolley. In 2017, International Conference on Microelectronic Devices, Circuits and Systems (ICMDCS). https://doi.org/10.1109/ icmdcs.2017.8211723.

S. Kowshika SS. Madhu Mitha G. Varshini M, Megha V Lakshmi K. "IoT based Smart Shopping Trolley with Mobile Cart Application," *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, Coimbatore, India, 2021, pp. 1186-1189, doi: 10.1109/ICACCS51430.2021.9441866.

Smart Shopping Cart Market Size, Share & Forecast to 2030 (researchandmarkets.com), 2025.

Yewatkar A, Inamdar F, Singh R, Ayushya, Bandal A (2016) Smart Cart with Automatic Billing, Product Information, Product Recommendation Using RFID & Zigbee with Anti-Theft. Procedia Computer Science, Volume 79, pp. 793-800.